

万字详解 Agent Skills：是什么？怎么用？和 Prompt、MCP 有什么区别？

👤 Guide 📄 AI 应用开发 📖 约 4159 字 ⌚ 大约 14 分钟

2025 年初，Anthropic 在推出 MCP（Model Context Protocol）之后，进一步提出了 Agent Skills 的概念。背后的思路其实很清楚：连接性（Connectivity）与能力（Capability）应该分离。

很多开发者认为“只要提示词写得好，AI 就能帮我做一切”。但事实是：**Prompt 适合单次任务，Skills 才是构建可复用 AI 能力的正确做法。**

Skills 把 AI 应用从“个人技巧”拉到了“工程化”的层面。今天 Guide 就带大家彻底搞懂这个概念，聊清楚 Skills 的设计理念、与相关技术的本质区别，以及如何在实战中用好这个能力。本文接近 1.2w 字，建议收藏，通过本文你将搞懂：

1. 🌟 **Skills 是什么**：为什么说 Skill 是“延迟加载”的 sub-agent？它的核心机制——上下文注入和延迟加载是如何工作的？
2. 🌟 **Skills vs Prompt vs MCP vs Function Calling**：这四者的本质区别是什么？它们分别适用于什么场景？这是面试中的高频盲区。
3. 🌟 **优秀的 Skill 长什么样**：一个设计良好的 Skill 应该包含哪些要素？元数据、触发条件、执行流程如何设计？
4. 🌟 **项目实战**：如何在真实开发中用 Skills 固化代码规范、排查流程、Review 标准？如何把团队中的“隐性知识”变成可复用的 AI 能力？

Skills 是什么？

用一句话概括：**Skill 是一个用自然语言定义的、具有特定领域上下文（Domain Context）的逻辑指令集，本质上是通过延迟加载（Lazy Loading）优化 Token 消耗的 Sub-Agent（子智能体）。**

在团队协作中，很多“隐性知识”都在老员工脑子里，比如代码规范、排查流程、Review 标准。Skills 的核心价值，就是把这些隐性规则变成显性的文档（SOP），让 AI 能够自主阅读、理解并执行。

与传统编程不同，Skills 不强制规定每一步的代码逻辑，而是用自然语言将决策权下放给模型——模型通过 `load_skill()` 动态加载 `SKILL.md` 后，将其中定义的规则、流程和约束实时注入到推理上下文中，指导后续的工具调用和决策。这既保留了 Agent 处理不确定性的优势，又避免了纯代码编排的僵化。

为什么不用"基于 Function Calling 封装"？这个表述容易让人误以为 Skill 是某种 Function Calling 的语法糖。实际上，Skill 的核心机制是上下文注入——Agent 读取 Markdown 文档，把其中的规则和流程纳入推理上下文。Function Calling 只是 Agent 执行某些动作（如调脚本、查资源）时可能用到的底层手段，不是 Skills 本身的定义层。注意：load_skill() 是对"Agent 读取并激活 SKILL.md "这一过程的概念性描述，不同工具（Claude Code、Cursor 等）的实际触发方式会有差异。

关键机制：

- **延迟加载（Lazy Loading）**：元数据保持简短（通常远少于正文）常驻上下文，正文仅在触发时动态注入，避免挤占 Token
- **动态上下文注入**：不同于静态文档的"阅读"，Skills 是将规则实时注入推理上下文，直接影响模型决策

Skills 和 Prompt、MCP、Function Calling有什么区别？

这也是面试中常被问到的点，容易混淆：

1. Skills vs Prompt

维度	Prompt	Skills
本质	单次对话的文本指令	可持久化、可发现的能力单元
复用性	随对话上下文丢失，难以维护	标准化封装，跨项目、多场景复用
加载机制	全量载入（挤占 Token）	延迟加载（按需读取正文）

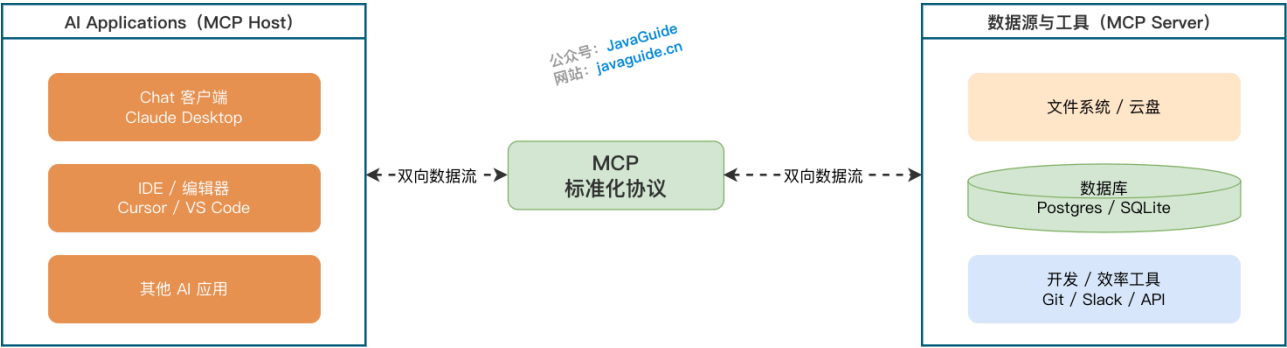
- **Prompt**：用户即时表达意图的载体（如"分析这份报表"）。
- **Skills**：包含元数据（何时使用）+ 正文（如何执行）的完整方案，通过 load_skill() 机制按需加载到上下文。

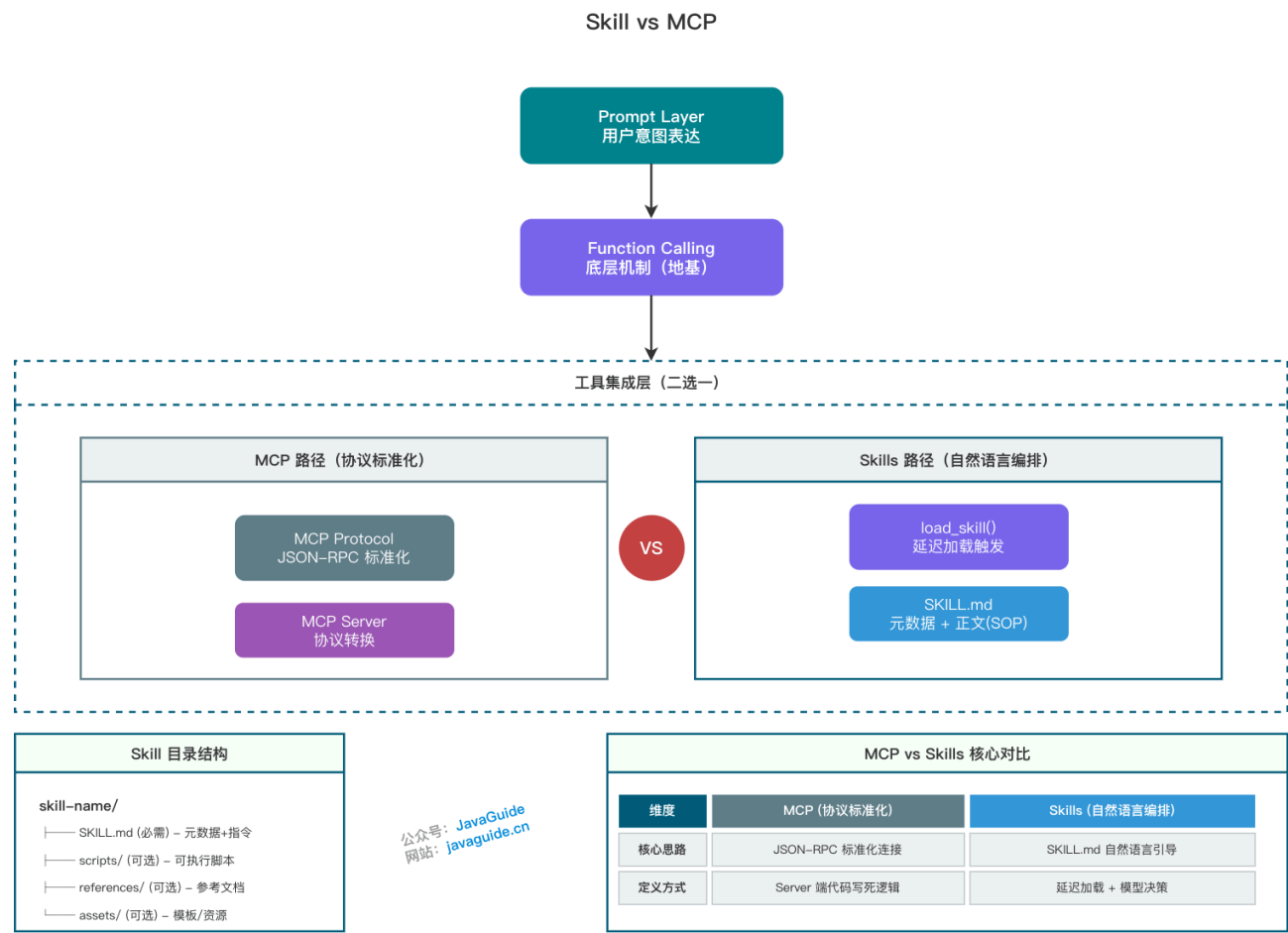
2. Skills vs MCP

这是最容易产生误解的地方。

维度	MCP (Model Context Protocol)	Skills
核心思路	标准化连接：通过 JSON-RPC 统一数据格式	逻辑编排：用自然语言描述复杂执行路径
定义方式	在 Server 端用代码 (TS/Python) 写死逻辑	在 SKILL.md 中用自然语言引导模型决策
环境依赖	需要运行一个 MCP Server 进程	依赖可执行环境（如本地 Shell 或沙箱）
哲学	以协议为中心：一次编写，所有 AI 通用	以模型为中心：利用模型推理能力处理不确定性

- **MCP 解决的是连通性**：它像 USB-C，让 AI 能以统一格式读文件、查数据库。
- **Skills 解决的是编排逻辑**：它像一份说明书，告诉 AI 如何执行复杂任务流——这些任务完全可以包括调用多个 MCP 工具。
- **两者的关系**：它们解决的是不同层面的问题。MCP 负责把外部系统接入进来，Skills 负责决定什么时候用、怎么组合这些能力。一个高级 Skill 的底层往往就是调用多个 MCP 工具。





3. Function Calling vs Skills

维度	Function Calling	Skills
层级	底层机制	上层应用
依赖关系	基础能力	在执行时可能使用 Function Calling（如加载文档、执行脚本、读取资源）
粒度	原子操作（单次工具调用）	复合流程（多步骤决策 + 工具组合）

Skills 没有创造新能力，而是通过自然语言文档将能力组织成更易用的形式：

1. Agent 读取 SKILL.md，将规则和流程注入推理上下文。
2. 根据上下文指导，Agent 可能通过 Function Calling 执行脚本、读取资源或调用 MCP 工具。

系统总结：

组件	一句话定义	形象类比	关键理解
Prompt	即时意图表达的载体	用户说的话	单次、易失
Function Calling	LLM 输出结构化调用的能力	神经信号	一切的基础，实现非结构化→结构化转换
MCP	标准化的工具接入协议	USB-C 接口	解决外部系统"如何接入"（连通性）
Skills	用自然语言定义的 sub-agent	任务说明书	解决复杂任务"如何编排"（执行逻辑），可调用 MCP 工具

四层关系：Function Calling 是地基 → Prompt 表达意图 → MCP 负责连通外部系统 → Skills 负责编排复杂任务流（可调用 MCP）

这里需要澄清一个常见误解：MCP 和 Skills 并不冲突，也不是非此即彼。

- **MCP** 解决外部系统如何接入：让 AI 能以统一格式读文件、查数据库、调用 API。
- **Skills** 解决复杂任务如何编排：用自然语言定义执行流程，这些流程完全可以包含调用多个 MCP 工具。

在实际项目中，两者经常配合使用：一个 Skill 的正文里会指导 Agent 先用 MCP 读取数据库，再用 MCP 调用外部 API，最后生成报告。

一句话总结：Prompt 承载意图，Function Calling 实现交互，MCP 负责连通外部系统，Skills 负责编排复杂任务流。

Skills 长什么样？你是怎么用的？

从结构上看，Skill 很简单，核心就是一个 SKILL.md 文件，包含元数据（描述什么时候用）和正文（具体的执行 SOP）。

设计上的亮点是“渐进式披露”：

- 元数据常驻上下文，AI 知道有哪些技能可用。
- 正文按需加载，只有触发时才读取，避免挤占 Token。

复杂点的 Skill，还会有附加的资源目录、脚本和参考文档。

Skill 的完整目录结构是这样的：

```
1 skill-name/  
2 |— SKILL.md          # 必需：元数据（何时使用）+ 正文（指令、流程、示例）  
3 |— scripts/          # 可选：可执行脚本（Python/Bash），按需调用  
4 |— references/        # 可选：参考文档，按需读取  
5 |— assets/           # 可选：模板、图片等资源
```

项目实战：

我在项目中主要用 Skills 来**固化工程标准**。比如定义一个 `code-reviewer` Skill，明确要求从架构合理性、异常处理完整性、日志规范、安全风险、性能隐患等多个维度进行结构化审查。这样 AI 在 Review 代码时，就会严格执行团队标准，而不是“随缘点评”。这对于保持代码质量的一致性非常有用。

除了 Code Review，我也会定义其他 Skill，例如：

- `api-endpoint-generator` - 按项目统一响应结构与异常模型生成标准化接口代码
- `database-access-review` - 审查数据库访问逻辑，关注索引使用与慢查询风险
- `refactor-analysis` - 先评估影响范围与依赖关系，再输出分步骤重构方案
- `security-audit` - 扫描 SQL 拼接、XSS、权限绕过等常见安全风险

优秀 Skill 示例：

- Code-Review-Expert（专家代码审查 Skill，以资深工程师视角进行结构化代码审查，覆盖：架构设计、SOLID 原则、安全性、性能问题、错误处理、边界条件）：
<https://github.com/sanyuan0704/code-review-expert>
- Git Commit with Conventional Commits（一个基于 Conventional Commits 规范的智能提交工具，可自动分析 diff、智能暂存文件并生成语义化 commit message，安全高效完成标准化 Git 提交）：
<https://github.com/github/awesome-copilot/blob/main/skills/git-commit/SKILL.md>
- TDD（测试驱动开发，先编写测试用例，观察它是否失败，然后编写最少的代码使其通过测试）：
<https://github.com/obra/superpowers/blob/main/skills/test-driven-development/SKILL.md>

<https://skills.sh/> 这个网站上可以查找自己需要和热门的 Skills。

▲ / SkillsAuditsDocs

SKILLS LEADERBOARD

Q Search skills ... /

All Time (81,445) Trending (24h) Hot

#	SKILL	INSTALLS
1	find-skills vercel-labs/skills	368.6K
2	vercel-react-best-practices vercel-labs/agent-skills	181.0K
3	web-design-guidelines vercel-labs/agent-skills	139.4K
4	remotion-best-practices remotion-dev/skills	119.6K
5	frontend-design anthropics/skills	112.8K
6	azure-ai microsoft/github-copilot-for-azure	94.1K
7	azure-observability microsoft/github-copilot-for-azure	94.0K
8	azure-cost-optimization microsoft/github-copilot-for-azure	94.0K
9	azure-storage microsoft/github-copilot-for-azure	93.9K
10	azure-deploy microsoft/github-copilot-for-azure	93.9K
11	azure-diagnostics microsoft/github-copilot-for-azure	93.9K

这里 Guide 多提一下，回答这个问题的时候，你也可以说自己团队用到了一些开源的软件开发 Skills 集合，例如 Superpowers 中内置的。

superpowers / skills 技能 /

Add file ...

obra

Enforce brainstorming workflow with hard gates and process flow

7f2ee61 · 2 weeks ago

History

Name	Last commit message	Last commit date
..		
brainstorming	Enforce brainstorming workflow with hard gates and proces...	2 weeks ago
dispatching-parallel-agents	Fix skill descriptions: remove workflow summaries that over...	3 months ago
executing-plans	docs: change main branch red flag to require explicit user c...	last month
finishing-a-development-branch	Improve skill cross-references for clarity and compliance	5 months ago
receiving-code-review	Add GitHub thread reply guidance to receiving-code-review	3 months ago
requesting-code-review	Fix skill descriptions: remove workflow summaries that over...	3 months ago
subagent-driven-development	docs: change main branch red flag to require explicit user c...	last month
systematic-debugging	Fix skill descriptions: remove workflow summaries that over...	3 months ago
test-driven-development	Fix skill descriptions: remove workflow summaries that over...	3 months ago
using-git-worktrees	docs(using-git-worktrees): add subagent/executing-plans ...	last month
using-superpowers	Enforce brainstorming workflow with hard gates and proces...	2 weeks ago
verification-before-completion	Standardize skill frontmatter names to lowercase and keba...	5 months ago
writing-plans	fix(writing-plans): use 4-backtick fence for nested code blo...	3 weeks ago
writing-skills	Fix stale Codex skills path in writing-skills SKILL.md	3 weeks ago

另外，很多 AI 编程 CLI 和 IDE 也会内置一些开箱即用的 Skills，例如 Claude Code 就内置了：

技能	功能	特点
<code>/simplify</code>	审查最近修改的文件（复用、质量、效率），自动修复	并行多代理审查，适合功能/修复后清理
<code>/batch <指令></code>	大规模批量修改代码库	自动任务拆分，每个任务在隔离 git worktree 中执行，可批量 PR
<code>/debug [描述]</code>	排查当前 Claude Code 会话问题	读取 debug log

如何编写高质量的 AI Agent Skills?

很多开发者第一次接触 Skills 时，会下意识地把它当成"文档"来写——堆砌背景介绍、安装指南、版本历史.....结果发现 AI 要么"读不懂"，要么"不用它"。

编写高质量的 Skills 是一项专门的技能——你写的不是给人看的 README，而是给 AI 写执行协议。这个区别决定了你需要完全不同的思维方式：

- **写给人**：注重可读性、完整性、背景知识
- **写给 AI**：注重精准性、可执行性、上下文效率

接下来的内容将系统性地介绍如何编写高质量的 Skills。这些原则来自 Anthropic 官方文档和社区大规模生产实践，经过实战验证。

语义精确的 Metadata（元数据）

Metadata 是 Agent 进行任务路由的核心依据，尤其是 description，它充当 LLM 的“索引”。

- **原则**：消除歧义，明确边界，并融入意图触发词。
- **优化逻辑**：从“描述功能”转向“定义场景、问题和触发条件”。

维度	不好的示例	优化的示例	说明
描述	分析系统日志	诊断 Spring Boot 生产环境的运行时异常，包括解析 Java 堆栈跟踪、定位 OOM 内存溢出和分析慢接口耗时。	边界清晰，避免泛化。
触发意图	无明确引导	当用户提到“接口报错”、“系统卡死”、“频繁 Full GC”或粘贴错误日志时，立即激活此技能。	提供具体触发词，便于 Agent 匹配。

在 Metadata 中添加 parameters 字段，定义输入输出格式（如 YAML），帮助 LLM 减少幻觉。例如：

1

2

```
parameters:
  input: { type: string, description: "错误日志或堆栈跟踪" }
```

yaml

3

```
output: { type: json, description: "诊断结果, 包括根因和建议" }
```

模块化与单一职责

大型“全能”Skills 会导致 LLM 在参数构建时产生幻觉。Agentic Workflow 更适合细粒度工具矩阵。

- **原则：**按排查维度拆分，确保每个 Skill 单一职责（SRP）。
- **优化方案：**避免单一“系统故障排查器”，改为工具集：
 - `jvm-metrics-analyzer`：专责通过 Prometheus 采集 JVM 指标（如堆内存、线程数）。
 - `distributed-trace-finder`：利用 SkyWalking 或 Zipkin 追踪特定 TraceId 的链路耗时。
 - `k8s-pod-event-viewer`：专责查询 Kubernetes Pod 状态变更和重启记录。

确定性优先原则

对于需要严谨逻辑的计算或格式转化，永远不要相信 LLM 的“直觉”，要让它去驱动脚本。

- **原则：**LLM 负责提取参数，脚本负责逻辑闭环。
- **案例优化：**当 Agent 发现 CPU 负载过高时，不要让它“盲猜”哪个线程有问题，而是让它调用一个封装好的诊断脚本。

Skill 定义中的执行逻辑：

“如果 CPU 使用率超过 80%，请提取节点 IP，调用 `./scripts/capture_thread_dump.sh`。不要尝试在对话框中手动模拟线程分析，直接解析脚本返回的 **Top 3 耗时线程堆栈**。”

渐进式披露策略

避免“信息过载”导致 Agent 迷失。通过文档的分层结构，让 Agent 只在需要时加载细节。

三层结构建议：

1. **SKILL.md**（主体）：定义核心故障类型（4xx, 5xx）和标准排查流转（SOP）。
2. **troubleshooting-guide.md**（附加）：放置一些罕见的“陈年老坑”或特定中间件（如 RocketMQ）的配置盲区。
3. **runbooks/**（数据文件）：存储历史故障知识库，由 Agent 通过 RAG 检索后再参考，而不是一股脑塞进上下文。

总结

编写高质量 Skills 的 五大核心原则：

原则	核心思想	关键实践
语义精确	从”描述功能”到”定义场景”	用祈使句 + 触发关键词 + 明确边界
极简主义	上下文是公共资源	删除噪音，10 行示例代替100行文字
模块化	单一职责避免幻觉	按排查维度拆解，而非建立”全能工具”
确定性优先	识别”脆弱操作”	LLM 提取参数，脚本负责逻辑闭环
渐进式披露	按需加载，避免上下文爆炸	L1 元数据常驻 + L2 正文按需 + L3 资源隔离

记住：Skills 本质上是**执行协议**，别把它当文档写。

总结与选型建议

核心观点

Skills 和 MCP 代表了智能体技术栈中两个关键的抽象层：

组件	一句话定义	形象类比	关键理解
MCP	标准化的工具接入协议	USB-C 接口	解决外部系统”如何接入”（连通性）
Skills	用自然语言定义的 sub-agent	任务说明书	解决复杂任务”如何编排”（执行逻辑）

两者是**互补关系**：

- MCP 专注于”能力”（提供基础设施连接）

- Skills 专注于"智慧"（提供业务逻辑和领域知识）

实践建议

场景	推荐方案	原因
外部服务连接（数据库、API、云服务）	优先使用 MCP	标准化接口，易于维护
复杂工作流（多步骤任务、领域专业知识）	优先使用 Skills	封装领域知识，可复用
上下文受限场景（长对话、大量工具）	使用 Skills 进行渐进式管理	降低 token 消耗 90%+
企业级智能体构建	采用 MCP + Skills 的分层架构	关注点分离，易维护扩展

面试准备要点

高频问题：

1. Skills 是什么？ → 延迟加载的 sub-agent，解决"如何编排"问题
2. Skills 和 MCP 的区别？ → MCP 负责连通性，Skills 负责执行逻辑，互补关系
3. 如何降低 token 消耗？ → 渐进式披露：元数据常驻，正文按需加载
4. 什么是渐进式披露？ → 三层架构：元数据 → 正文 → 附加资源
5. 如何编写高质量 Skills？ → 精准 description + 单一职责 + 确定性优先

追问准备：

- 你的团队用了哪些 Skills？如何组织的？
- 如何评估一个 Skill 的好坏？
- Skills 如何与 MCP 配合使用？
- 如何避免 Skills 的上下文污染问题？